

[Skip to main content](#) [./ClaudeCode](#) [X](#) [Search in r/ClaudeCode](#)



Create



r/ClaudeCode • 5mo ago
Azrael_666

...

How are you actually using Claude Code as a team? (not just solo)

Help Needed

So for the past two months I've been using Claude Code on my own at work and honestly it's been great. I've built a ton of stuff with it, got way faster at my job, figured out workflows that work for me, the whole thing.

Now my boss noticed and basically said "congrats, you're now in charge of AI transformation for the product team." He got us a Team subscription, invited 5 people, and wants me to set up shared workflows, integrate Claude Code across our apps, etc...

The problem is: everything I know about Claude Code is from a solo perspective. I just used it to make myself more productive. I have no idea how to make it work for a team of people who have never touched it.

Some specific things I'm trying to figure out:

- How do you share context between team members? Like if I learn something important in my Claude Code session, how does that knowledge get to everyone else? Right now the best I've found is the [CLAUDE.md](#) file in the repo but curious if people are doing more than that
- For those on Team plans, how are you actually using Projects on claude.ai? What do you put in the knowledge base? Is it actually useful for a your team?
- How do you onboard people who have never used Claude Code? I learned by watching YouTube and reading Reddit for weeks which is not exactly a scalable onboarding plan lol
- Is anyone actually doing the whole "automated workflows" thing? Like having Claude post to Slack, create tickets, generate dashboards? Or is that more hype than reality right now?
- How do you keep things consistent? Like making sure Claude gives similar quality output for everyone on the team and not just the one person who knows how to prompt it well

I feel like there's a huge gap between "I use Claude Code and it's awesome" and "my whole team uses Claude Code effectively" and I'm standing right in that gap.

Would love to hear what's actually working for people in practice, not just what sounds good in theory. What did you try that failed? What surprised you?

54

42

1



hyperagentapp • Promoted

...

Skip to main content

Create

Sign Up

hyperagent.com

Editor & Quality Gate	25	100%	100%	100%	\$10.00	10 minutes
Codebase Director	24	100%	100%	100%	\$10.00	10 minutes
Product Producer	23	100%	100%	100%	\$10.00	10 minutes
Global Media Manager	8	100%	100%	100%	\$7.00	10 minutes
SEO & Distribution Strategist	26	100%	100%	100%	\$14.00	10 minutes
Workflow Engine	29	100%	100%	100%	\$20.00	10 minutes
Community Manager	26	100%	100%	100%	\$10.00	10 minutes
Newsletter Architect	9	100%	100%	100%	\$5.00	10 minutes
Talent Scout	4	100%	100%	100%	\$3.00	10 minutes
Comprehensive Intel Agent	22	100%	100%	100%	\$10.00	10 minutes

Join the conversation

GIF

Aa

Cancel

Comment

Sort by: Best Search Comments



thlandgraf • 5mo ago

Been through this exact transition. The biggest wins: treat [CLAUDE.md](#) as team infrastructure not personal notes — build commands, naming conventions, testing patterns, all version-controlled in Git so every session picks it up. And custom skills in `.claude/commands/` solve the consistency problem — instead of hoping everyone prompts the same way, write a markdown file for each repeatable workflow and anyone can run `/my-skill` to get consistent output. For onboarding, pair sessions beat docs. Have people watch someone experienced for 20 minutes then swap. The hardest thing for new users is calibrating how much context to give and that's learned by watching, not reading.

27

Reply



HaagNDaazer • 5mo ago

I like the idea of buddy vibing, I'm definitely stealing that!

4

Reply



_Bo_Knows • 5mo ago

What I've found is pair coding is the way to go. I find it very similar to video gaming or sports. You can play around with the game and eventually learn how get success, or you can turn on Twitch or TV and see how the best people in the world use it. It gives you an idea of what's possible.

3

Reply



theangi • 5mo ago

Sharing team plugins, with skills, commands etc seems the best way to share it across teammates. Oversimplifying, we're simply writing down on various `.md` files what has been done since ages without llms.

Unfortunately, there is no full standard across different AI tools, and this bounds pretty much to Claude only.

2

Reply

[Skip to main content](#)[+ Create](#) 1 [Reply](#) ...**rahat008** • 5mo ago

have you tried git aware coding agent? it think, pair your coding with the git aware subagent is perfect for newcomers. They don't need to know the whole picture only the agents needs that.

 1 [Reply](#) ...**Substantial_Doubt139** • 3mo ago

All of this matches what we've seen, especially the `.claude/commands/` point. Treating prompts as code that gets reviewed and versioned is the only way the consistency problem becomes tractable.

The one thing I'd add. [CLAUDE.md](#) and skills both work brilliantly for the "how we do things" layer, things like naming, testing, build commands. They're stickier for the "what we decided and why" layer, especially when a decision changes. A skill encodes the current right answer but doesn't tell you it used to be a different answer six weeks ago, or which work was built on the old assumption. Most teams I've seen using your setup hit that wall around month three, when someone reopens an old decision and there's no clean way to find what depends on it.

Not a knock on the approach, more that infrastructure-as-code handles the standing rules better than it handles the mutable decisions underneath them.

 1 [Reply](#) ...**[deleted]** • 5mo ago**italian-sausage-nerd** • 5mo ago

There is a huge gap indeed, and the tech moves so fast it's hard to keep everyone together while you struggle to figure out what new features drop each week...

Anyway a single skills.md repo with "this is how we do auth", "this is how you should write test reports" etc., and a sync script helps enforce consistency across the sdlc, broader than what you could fit in a single projects claude.md

 6 [Reply](#) ...**Such_Independent_234** • 5mo ago

Related to agent output quality, I'm finding that the best things you can do for agents was the same thing you should have been doing for humans all along. I'm not sold on the hyped tool of the day or MCP that promises you the best agent memory ever. Constrain agents through tools, environment, and code organization.

I think the patterns that survive AI assisted development are the ones agents can't ignore. Things like linter errors, type errors, permission boundaries, CI gates, etc. These are deterministic. Relying on agent specific

[Skip to main content](#)[+ Create](#)[4](#) [Reply](#) [...](#)[u/useblacksmith](#) • Promoted

Blacksmith: so fast you can touch grass.

[Learn More](#)[blacksmith.sh](#)[ultrathink-art](#) • 5mo ago

Senior Developer

Team usage unlocked a weird scaling dynamic for us — the bottleneck shifted from 'how fast can one person code' to 'how do multiple agents stay coherent on the same codebase.'

We run 6 AI agents that commit code daily. The coordination problem is harder than the coding problem. Each agent has its own CLAUDE.md with role-specific instructions, but the real work is making sure a design agent and a coder agent don't step on each other's changes.

What's worked: task states (pending → claimed → in_progress → review → complete) enforced at the work queue level. No agent can grab work another agent already claimed. Heartbeats detect dead agents and reset their tasks.

What we got wrong early: shared context. Agents reading each other's in-progress notes would sometimes contradict or overwrite. The fix was strict artifact handoffs — agent A produces a JSON spec, agent B consumes it. No shared mutable state between runs.

Curious if your team pattern is sequential (review each other's work) or truly parallel.

[4](#) [Reply](#) [...](#)[kochamisenua](#) • 5mo ago

So ai swarm

[2](#) [Reply](#) [...](#)[kochamisenua](#) • 5mo ago

Do you keep your own tester role? In my experience such setup only shines when the agent can test themselves e2e before claiming that the task or user story is done.

I am doing something similar with the planner and intake (like overall user story), planner splits that into tasks and those small coding tasks can be implemented separately (I added a blocked column, blocked tasks are moved to planned once they are unblocked)

[Skip to main content](#)[+ Create](#) 1 [Reply](#) ...**HaagNDaazer** • 5mo ago

Also worth enforcing that everything must be a Pull Request as breadcrumbs for the project.

I am also thinking through this and am also using Linear issues as a shared history of tech decisions across the project that Claude can then search to find related issues and learn as much as it can from that to ask better clarifying questions during planning. Then Claude takes the linear issue through the whole process, updating status and leaving comments along the way as it works so there is a nice history per issue.

Lastly, for most of the Claude type markdown files, you can also have a .local version that is not git versioned, giving each team member a way to customize aspects for themselves, those changes should be reviewed regularly to see where individuals are maybe improving on a process and potentially merge that into the team wide markdowns

 3 [Reply](#) ...**ObjectiveSalt1635** • 5mo ago

Pull request mandatory also helps enforce using code review ai like code rabbit, Claude, codex etc

 1 [Reply](#) ...**HaagNDaazer** • 5mo ago

Exactly!

 2 [Reply](#) ...**MagicaNexus9** • 5mo ago

Is there a difference between linear and GitHub issues ?

 1 [Reply](#) ...**HaagNDaazer** • 5mo ago

issues in Linear are more of your actual task tickets that you are working on. GitHub issues are bugs people are reporting

 1 [Reply](#) ...**__mson__** • 5mo ago
Senior Developer

It doesn't have to be that way. I use GitLab Issues for planning and tracking work. It's no different than working with Jira Tickets, just a different tool.

Plus you get native integration to track changes from commits to MRs, and back to Issues if you need extra context about a change in the future.

[Skip to main content](#)[+](#) Create

Ah yeah that is totally valid



1



Reply



boatsnbros • 5mo ago

Yep about a month ago we built an internal Claude plugin based on our development standards docs + a few popular plugins (eg superpowers), and accept PRs to it for adding new skills for workflows, and also keep a Claude.md along side the readme.md for all repos. PRs now require notes from our /review-code skill and full tdd proof. Biggest change has been we no longer want individual developers working on the same microservice at the same time - too much merge conflict heck as writing lines of code is no longer the slow part. Been great for seniors who run code reviews (as the 'brainstorm' + write-plan + execute-plan workflow checks existing patterns in codebase & plans) instead of juniors just saying 'add this feature' and getting something that works but is hilarious inconsistent w rest of codebase. Been great for juniors as they now had a framework of how to develop. Microservice architecture + Claude plugin has really unlocked productivity for us.



2



Reply



u/Amazon_Official • Promoted

You reddit right: More than 60% of sales on Amazon come from independent sellers.

[Learn More](#)

aboutamazon.com



ryan_the_dev • 5mo ago

Set up a marketplace. Use that for shared skills. Ignore customizing Claude md.

Standardize with skills.

Create good agentic workflows to super charge developers.

Think about things like PR review, story management, documentation, etc. automate those boring tasks.

Another big thing we have built out is our on call stuff. Building out investigation skills that are specific to our product services. Knows how to search out logs and metrics across services to correlate things.

Here is an example of a agentic coding workflow my team is playing around with. Based it off software engineering books.

<https://github.com/ryanthedev/code-foundations>

[Skip to main content](#)[+ Create](#)

Yeah shared skills has been the main tool that's worked for us as well.

1 [Reply](#) ...



malek_hor30 • 5mo ago

I was making a conversation with Claude about this issue, since I have the same challenge. Here what I got at the end: <https://claude.ai/public/artifacts/e220c911-6063-495d-bf50-1383c5f4fcd7>



mbcoalson • 5mo ago

I'm just getting started on this as well. My team is primarily mechanical engineers, not SWEs. My plan is to put my team on Claude Cowork, not Claude Code. Then I start pushing plugins out onto the private marketplace you have available on the Team account. Plugins combine commands, skills, and hooks as needed and can be versioned by the admin, which should be you.



treasury_minister • 5mo ago

I find this interesting. I'm thinking the same approach for my team



mbcoalson • 4mo ago

Note hooks do not currently work within Cowork.



Fabian-88 • 4mo ago

wie läuft es bisher? wir sind R&D scientists, also auch no-coder. Wo kommst du her? Sollen wir mal in austausch treten?



mbcoalson • 4mo ago

Does your work have an overlap with Energy Efficiency in commercial buildings?



Fabian-88 • 4mo ago

ah sorry for german, seems like reddit was auto translated.
I work in the research and development for a food company.



[Skip to main content](#)[+](#) Create

Take a look at the Marvin skill as I stopped using the desktop app with it



paulcaplan • 5mo ago

The most helpful thing is pairing sessions



freeformz • 5mo ago

Add Claud.md and rules to repos used by folks Setup an internal marketplace with shared skills/plugins/etc. I recently did this.



ultrathink-art • 5mo ago

Senior Developer

The transition from solo to team use exposes something that barely matters when you're alone: coordination overhead.

Solo, you can be pretty undirected — throw problems at it, iterate fast. Team use requires shared context and explicit agreements about what each agent is responsible for. CLAUDE.md discipline becomes the only coordination layer that actually sticks.

We run 6 Claude Code agents in production — design, code, marketing, ops, security — all running concurrently. The thing that broke most often wasn't individual agent quality. It was git conflicts and deploy races when multiple agents pushed to main simultaneously. We had to add serialization rules before any of it was stable.

Before adding your first teammate: figure out your merge/deploy serialization strategy first. It'll save you a painful incident.



tbanas • 5mo ago

So, do you guys actually use Claude's Team plan?



elmarto356 • 5mo ago

in my team have use google drive, using the internal tools, we have think about use of github, but the problem is now we work with PDFs, Opus (whatsapp audio format), MP3, MP4, etc, and is difficult see or upload this files to github, and my team not all have technical knowment



Create



wie hat das für euch funktioniert mit google drive? hab das selbe gedankliche problem mit github, da wir no-coder sind.



Shoddy-Brilliant4893 • 4mo ago

[u/Azrael_666](#) , from you post, you're mainly framing the problem of not having a Universal Orchestration layer that allows your team to be in control while executing at scale, correct? I think having the perfect [claude.md](#) will not be sufficient



petertanham • 3mo ago

You might find <https://SharedContext.ai> for sharing context (as the name suggest!).

It's an internal skills library for teams to share best practices - which is also a great way to onboard people too